

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

Национальный исследовательский ядерный университет «МИФИ»

АНАЛИТИЧЕСКИЙ ОТЧЁТ

по курсовой работе:

Применение методов машинного обучения для прогнозирования эффективности химических соединений против вируса гриппа

Выполнил:

студент группы М25-525

специализации «Машинное обучение»

Карсаков Степан Викторович

Москва – 2026

Оглавление

Введение	3
1 Исходные материалы и теоретическая база	4
1.1 Учебный материал	4
1.2 Описание датасета	5
1.3 Постановка задачи	5
2 Архитектура приложения	6
2.1 Общие принципы	6
2.2 Анализ кода	7
2.3 Структура проекта	8
2.4 Иерархия классов задач	8
2.5 Запуск	9
3 EDA	10
3.1 Базовые характеристики датасета	10
3.2 Проверка определения SI	10
3.3 Распределения целевых переменных	11
3.4 Парные зависимости между таргетами	12
3.5 Корреляционный анализ признаков с таргетами	13
3.6 Фильтрация признаков	14
3.7 Баланс классов для задач классификации	14
3.8 Область применимости (Applicability Domain)	15
3.9 Сводка решений по EDA	15
4 Инструментарий и методология моделирования	16
4.1 Используемые библиотеки	16
4.2 Состав моделей-кандидатов	16
4.3 Единый пайплайн обучения	17
5 Результаты регрессионных моделей	18
5.1 Регрессия IC50	18
5.2 Регрессия CC50	18

5.3	Регрессия SI	19
6	Результаты классификационных моделей	20
6.1	Классификация: IC50 выше медианы	20
6.2	Классификация: CC50 выше медианы	20
6.3	Классификация: SI выше медианы	21
6.4	Классификация: SI > 8	21
7	Сравнительный анализ и обсуждение	22
7.1	Регрессия против классификации	22
7.2	Линейные модели против ансамблей	22
7.3	Согласованность EDA и финальных результатов	22
7.4	Нюансы	23
7.5	Рекомендации по развитию	23
	Заключение	24

Введение

Процесс создания нового лекарственного препарата является сложным и многоэтапным. Необходимо определить его химическую формулу, синтезировать соединение, провести первичные биологические испытания и организовать тестирование. Все эти этапы требуют значительного времени, однако современные методы машинного обучения способны существенно ускорить данный процесс: с помощью построенных моделей можно прогнозировать эффективность соединений и сужать пространство кандидатов для дальнейшего синтеза.

В рамках этой работы рассмотрена практическая ситуация: химиками были предоставлены конфиденциальные данные о 1000 химических соединений с указанием их эффективности против вируса гриппа. Параметры, характеризующие эффективность, обозначаются как IC50, CC50 и SI. Селективный индекс значение SI рассчитывается на основе параметров IC50 и CC50: $SI = CC50 / IC50$ [1].

Цель работы — на основании предоставленных данных от химиков построить прогноз, позволяющий подобрать наиболее эффективное сочетание параметров для создания лекарственных препаратов.

Для достижения цели последовательно решались следующие задачи:

- изучить теоретическую основу по предоставленному курсу лекций МГУ;
- спроектировать архитектуру Python-приложения с разделением ответственности и единым пайплайном для всех задач;
- провести разведочный анализ данных (EDA) и предобработку;
- построить и сравнить модели регрессии для IC50, CC50 и SI;
- построить и сравнить модели бин. классификации по четырём порогам;
- сравнить полученные результаты, выполнить анализ и обосновать выбор наиболее качественного решения.

Глава 1

Исходные материалы и теоретическая база

1.1 Учебный материал

В качестве основного учебного материала использовался конспект лекций "ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ В ХИМИИ И МАТЕРИАЛОВЕДЕНИИ" составленный командой студентов проекта "TeachInMSU" (МГУ им. М. В. Ломоносова) [2], по лекциям Митрофанова А.А. Документ, охватывает как общие вопросы машинного обучения, так и специфику его применения в химии и материаловедении.

В рамках настоящей работы полезными оказались следующие разделы:

- Занятие 1 — введение в задачи МО: регрессия, классификация; необходимость кросс-валидации; понятие области применимости модели;
- Занятие 5 — молекулярные дескрипторы: классификация 0D/1D/2D/3D/4D, библиотека rdkit, фингерпринты Морган и Masscs (для этой работы практической полезности никакой не принесло,, но дало общее представление, как примерно "генерились" данные);
- Занятие 6 — классические алгоритмы: деревья решений, Random Forest, линейная и логистическая регрессии, KNN, метрики MAE, MSE, R^2 ;
- Занятие 10 — автоматический подбор гиперпараметров: Tree Parzen Estimator (почему-то в конспект TRE).

В дополнение к конспекту использовались материалы с хабра:

- современный гайд по библиотеке scikit-learn [6] — опорный материал по сборке пайплайнов и подбору параметров;
- обзор библиотеки Optuna и алгоритма Tree Parzen Estimator [4] — использовался при настройке автоматического подбора гиперпараметров;
- материал по особенностям работы XGBoost [5];

- статья об ML-скрининге антибиотиков [7] — как пример практической пользы подобных моделей: модель сужает пространство кандидатов по предсказанным значениям IC50, после чего соединения проверяются уже лабораторно;
- справочный материал компании Creative Diagnostics о тестах CC50/IC50/SI [1] — использовался для понимания смысла переменных и для интерпретации порога SI.

Из источника [1] были взяты предметные определения, на которые опирается работа: **CC50** — концентрация вещества, при которой гибнет 50% клеток (мера токсичности для клеток); **IC50** — концентрация, при которой подавляется 50% вирус-индуцированного эффекта (мера противовирусной активности); **SI** — отношение CC50/IC50, чем оно выше, тем перспективнее соединение. Там же приводится практический ориентир: соединения с $SI \geq 10$ обычно считают активными *in vitro*; (у нас по условию задания используется чуть порог чуть мягче $SI > 8$).

1.2 Описание датасета

Исходный датасет — эксель док, экспортированный в CSV-файл с разделителями. Размер: 1001 строка x 213 колонок, из которых 3 — целевые, 210 — дескрипторы. Данные — числовые (векторизация не нужна)

Целевые переменные:

- **IC50, mM** — ингибирующая концентрация (чем меньше, тем активнее соединение против вируса); (2)
- **CC50, mM** — цитотоксическая концентрация (чем больше, тем менее токсично для клеток);
- **SI** — селективный индекс (чем больше, тем перспективнее соединение).

210 дескрипторов разбирать глубоко не стал, хотя при разборе, из названий стало понятно что признаки получены чем-то типа rdkit (видимо из каких-то готовых наборов SMILES).

1.3 Постановка задачи

Согласно заданию, на основе данных нужно построить семь моделей: три регрессионных — для IC50, CC50 и SI, четыре классификационных — предсказание превышения медианы для каждого из трёх таргетов и предсказание значения $SI > 8$

Глава 2

Архитектура приложения

2.1 Общие принципы

Параллельное освоение материалов Л.Рамальё "Fluent Python-[3], позволило реализовать в проекте production-подобную архитектуру, закрепив пройденный материал в реальных задачах.

При проектировании применялись принципы:

- Разделение ответственности — каждый модуль решает одну задачу: загрузка, предобработка, описание моделей, оркестрация, EDA, отдельные задачи;
- Общий пайплайн через паттерн Template Method — абстрактный класс описывает общий алгоритм работы (загрузка - предобработка -сплит -подбор- оценка - сохранение), а под-классы с имплементацией переопределяют то, что отличается между задачами;
- Конфиг через dataclass — константы (random_state, пути, имена таргетов, параметры предобработки) вынесены в отдельный модуль config.py;
- Воспроизводимость — проект реализован как Python-пакет с метаданными в pyproject.toml; зависимости фиксируются через менеджер uv и файл uv.lock;
- Логирование — единый модуль logging_setup.py обеспечивает вывод с отметками времени и прогрессом обучения каждой модели .

Подходы, заимствованные из "Fluent Python":

- использование @dataclass для контейнеров данных без избыточного boilerplate (гл 5 - Data Class Builders);
- фабрики моделей в виде первоклассных функций, принимающих словарь гиперпараметров (гл.7 — Functions as First-Class Objects);
- абстрактный базовый класс на основе abc.ABC и @abstractmethod; (гл10 - Design Patterns, гл.13-Interfaces, Protocols, and ABCs)

2.2 Анализ кода

Пакет прогонялся через `ruff` и `муру` — стандарт для пайтон-проектов. Оба инструмента лежат в дев-группе `pyproject.toml` и ставятся отдельно от `runtime`:

- `ruff` — линтер-форматтер: ошибки-мусор в коде, импорты, подозрительные конструкции типа `lambda` в циклах;
- `муру` — проверка аннотаций типов по пакету (тут не особо был нужен - скорее для порядка. Хотя один баг выловлен: в `run_all` список `ALL_TASKS` был без нормальной типизации - можно было создать абстрактный `BaseTask`. Пофикшено через `TaskClass`).

Проверка из `chemical_coursework/`:

```
uv sync --group dev
uv run ruff check src
uv run ruff format --check src
uv run муру
```


2.3 Структура проекта

```
chemical_coursework/
|-- pyproject.toml          # dependencies + entry-points (uv)
|-- uv.lock                #фикс.версий
|-- README.md
|-- .gitignore
|-- data/
|   `-- Данные_для_курсовой_Классическое_M0.csv
|-- docs/
|   `-- ai-in-chemistry-and-materials-science-M-3.pdf #конспект по видеолекциям
|-- artifacts/             #результаты после прогона
|   |-- figures/           # визуальщина EDA
|   |-- models/           # joblib
|   `-- metrics/          # метрики
`-- src/
    `-- chemical_coursework/
        |-- __init__.py
        |-- config.py      # константы и пути
        |-- logging_setup.py # конфиг логера
        |-- data_io.py     #утилита для чтения csvшки
        |-- preprocessing.py #фильтрация фич, удаление выбросов
        |-- base.py        # абстракции, базовые классы, модельки
        |-- model_zoo.py   # фабрики моделей + параметрические пространства
        |-- eda.py         # EDA
        |-- run_all.py     # прогон всех задач
        `-- tasks/        # задачи согласно условию
            |-- regression_ic50.py
            |-- regression_cc50.py
            |-- regression_si.py
            |-- classification_ic50_median.py
            |-- classification_cc50_median.py
            |-- classification_si_median.py
            `-- classification_si_gt8.py
```

Рис. 2.1: Структура проекта

2.4 Иерархия классов задач

Базовый абстрактный класс `BaseTask` определяет метод `run()`, который выполняет общий пайплайн для любой задачи. Класс `BaseRegressionTask` переопределяет стратегию кросс-валидации

(KFold), метрику для Optuna (`neg_root_mean_squared_error`) и набор финальных метрик (MAE, RMSE, R^2). Аналогично, `BaseClassificationTask` использует `StratifiedKFold`, метрику `roc_auc` и набор метрик классификации (`accuracy`, `precision`, `recall`, `f1`, `roc_auc`).

Каждая из семи таск-классов наследуется от соответствующего базового и оверрайдит лишь то, что специфично для неё: имя таргета, способ получения целевой (пример: `np.log1p(y)` для регрессий или `(y > порог).astype(int)` для классификаций) и набор кандидатов из `model_zoo`.

Подобная архитектура позволила исключить (ну или значительно преуменьшить) дублирование кода и создать сущности которые, как говорится, ”открыты для расширения”(но к сожалению про python нельзя сказать - ”закрыты для изменений”).

2.5 Запуск

```
uv sync          # установка зависимостей
uv run chemcw-eda      # EDA результаты( в src/chemical_coursework/artifacts)
uv run chemcw-run-all  # прогон всего
```

Глава 3

EDA

EDA реализован в `eda.py` и запускается командой `uv run chemsw-eda`. Результаты сохраняются в `artifacts/`: графики — в `figures/`, метрики — в `metrics/eda_summary.json` и CSV-файлах.

3.1 Базовые характеристики датасета

Датасет содержит 1001 строку и 213 колонок (3 таргета + 210 признаков). Все столбцы — числовые: 107 типа `float64`, 106 типа `int64`. Пропуски есть, но минимум: 36 ячеек в 12 колонках. Логично залить медианой по столбцу, что и было сделано. Статистика по таргетам представлена в таблице 3.1.

Таблица 3.1: Описательная статистика целевых

Таргет	min	q1	median	q3	max	std
IC50	0.0035	12.5	46.5	224.97	4128.52	402.169
CC50	0.70	99.99	411.03	894.08	4538.9	642.9
SI	0.011	1.43(3)	3.846	16.56	15620.6	684.5

3.2 Проверка определения SI

Для контроля качества данных была проведена проверка соотношения $SI = CC50 / IC50$. Максимальная абсолютная разность между значением SI из датасета и расчётным составила $4.1 \cdot 10^{-4}$, медианная относительная разность — $1.3 \cdot 10^{-10}$; доля строк с относительной разностью менее 1% равна 100%.

Вывод. Селективный индекс реально является производной от двух других таргетов. Это означает, что регрессия SI в принципе не очень и нужна — её можно было бы вычислить из предсказаний CC50 и IC50. Но этого требует задание соответственно задача регрессии SI выполняется отдельно

3.3 Распределения целевых переменных

Распределения IC50, CC50 и SI в исходном масштабе (рис. 3.1 синий ряд) имеют явные правые хвосты. Лог преобразование $\log(1+y)$ приводит к более симметричному виду - зелёный ряд.

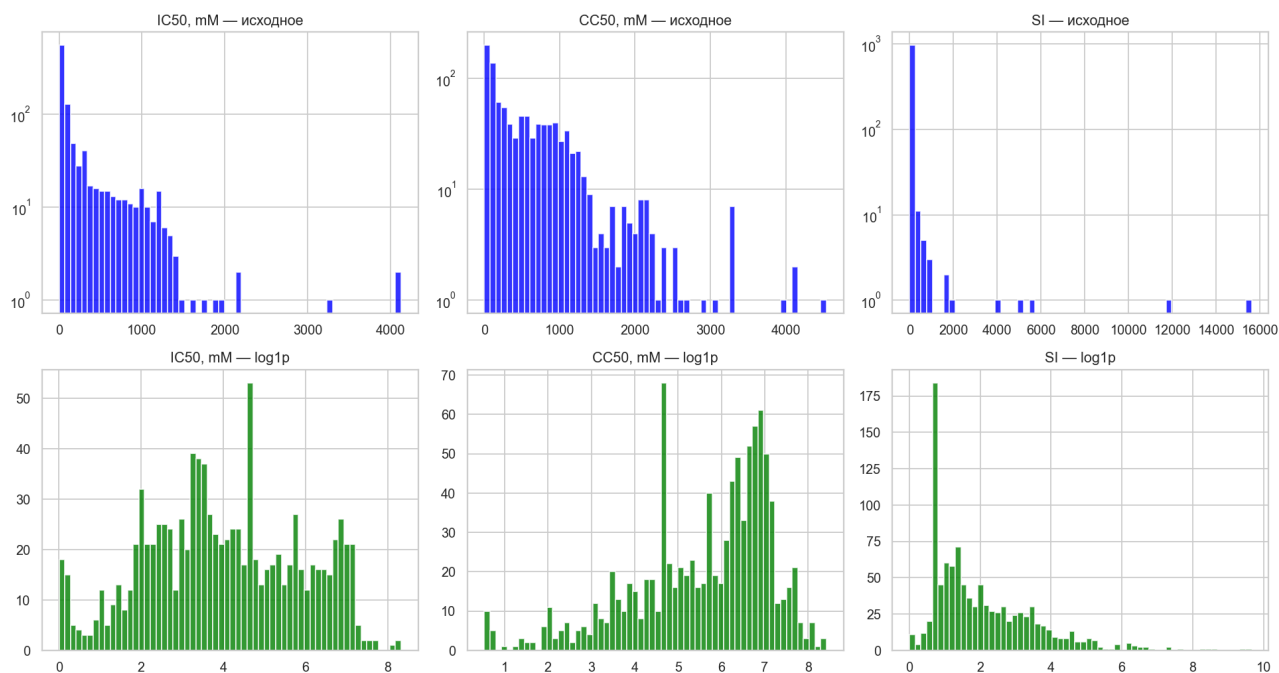


Рис. 3.1: распределения целевых до и после преобразования $\log1p$

Боксплоты подтверждают наличие большого количества выбросов - особенно для SI.

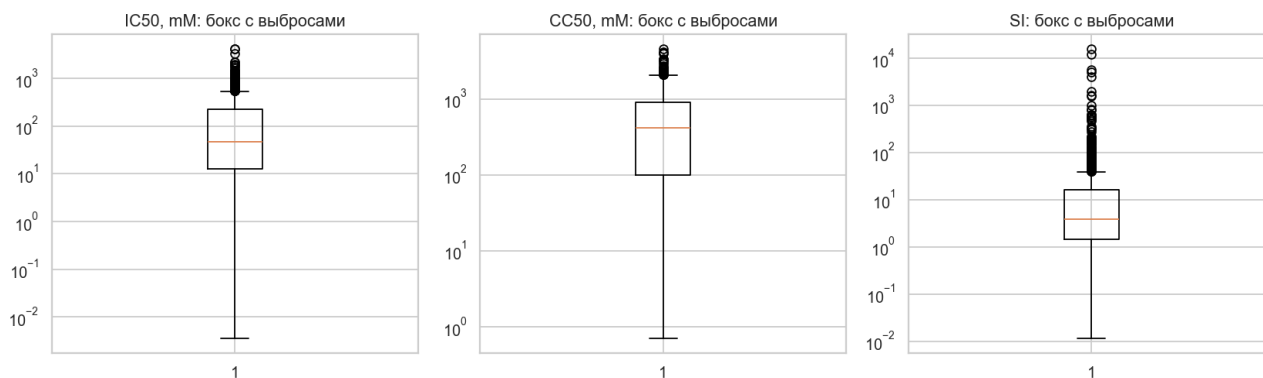


Рис. 3.2: боксплоты целевых переменных

Вывод: для регрессионных задач решено обучать на логарифмированных целевых переменных - это стабилизирует обучение, особенно для линейных моделей и снижает влияние единичных выбросов на функцию потерь. Ещё для регрессионных применяется фильтрация выбросов по правилу $3 \cdot IQR$.

3.4 Парные зависимости между таргетами

На парных графиках видно, что IC50 и CC50 связаны умеренно (чисто визуально - при большем значении одной переменной - в среднем больше и другой. И vice versa).

Формула $SI = CC50/IC50$, но по pairplot, само отношение не видно - только отдельные пары. Корреляция SI с IC50 - прослеживается хорошо. С CC50 - слабее.

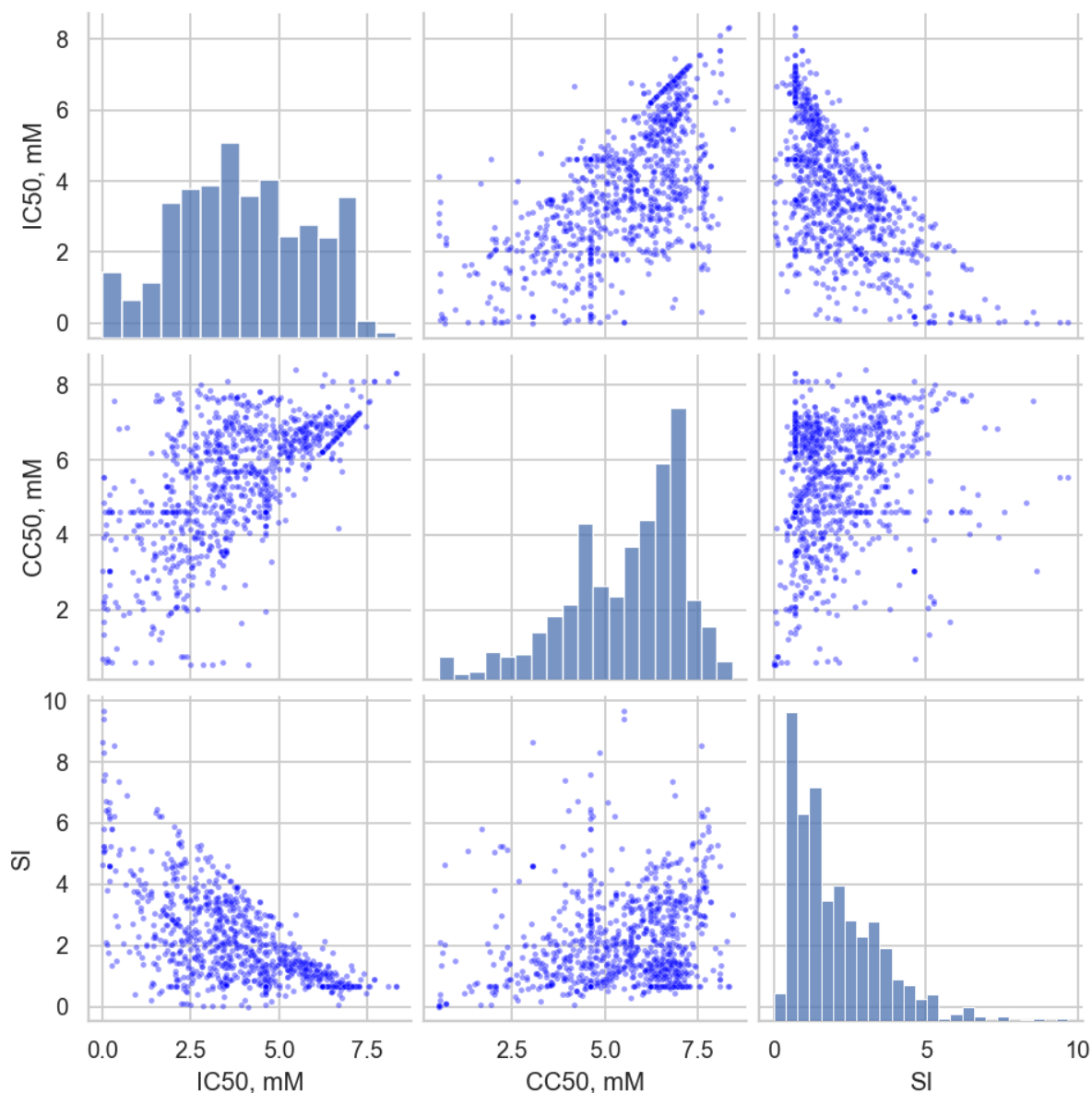


Рис. 3.3: парные графики

Вывод. Умеренная связь IC50 и CC50 при зависимости SI от обоих параметров, говорит о том что задача регрессии SI всё-таки сложнее - ошибки прогнозов базовых таргетов перемножаются в производной величине.

3.5 Корреляционный анализ признаков с таргетами

Построена тепловая карта, для 30 признаков с наибольшей корреляцией хотя бы с одним из таргетов. Макс значение корреляции одного признака с одним таргетом не превышает 0.27 (например, NumSaturatedHeterocycles для IC50, Kappa3 для CC50, FractionCSP3 для SI) значит ни один из признаков в одиночку не объясняет целевую в значимой мере.

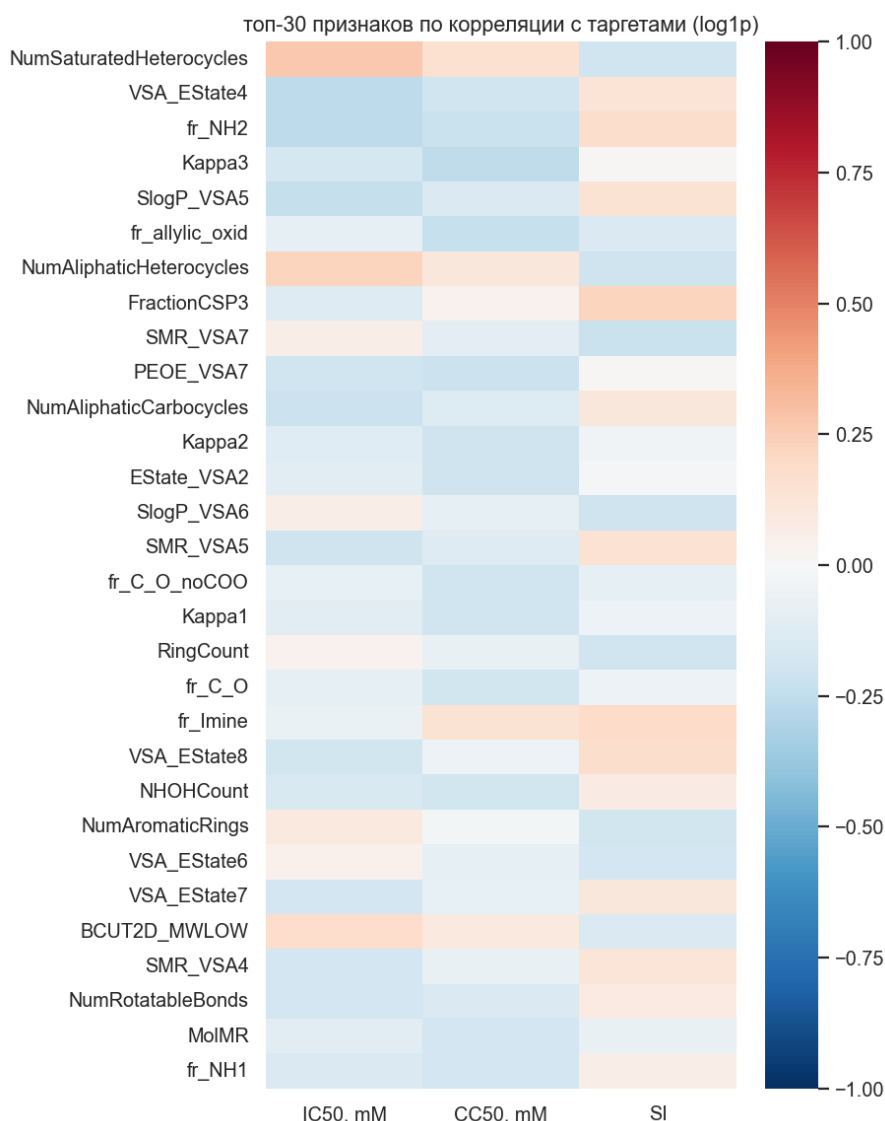


Рис. 3.4: Тепловая карта корреляций топ30 признаков с целевыми переменными

Вывод: связь признаков с таргетами нелинейная; это означает что качества стоит ожидать от моделей ансамблей на решающих деревьях - Random Forest, XGBoost [5] и LightGBM - они умеют ловить нелинейности и взаимодействия признаков. Линейные - Ridge, ElasticNet по сути бесполезны и добавлены как минимальный порог качества (просто увидеть, насколько ансамбли лучше)

(по факту так и получилось: во всех 7 задачах, "победили" деревья)

3.6 Фильтрация признаков

В предобработке реализовано:

- удаление пропущенных колонок;
- удаление колонок с низкой дисперсией - константы (по ЕДА - 18);
- удаление одного из членов каждой пары с $|r| > 0.95$ (по ЕДА - 33).

Результаты EDA: всего: пустых колонок — 0, констант — 18, скоррелированных — 33. В результате остаётся 159 признаков. В пайплайне моделей фильтрация повторяется после удаления выбросов по таргету, поэтому число оставшихся признаков для разных задач чуть колеблется (157-159).

Вывод: удаление мусорных и избыточных признаков ускоряет обучение и стабилизирует линейные модели.

3.7 Баланс классов для задач классификации

Распределение положительного класса для четырёх классификационных задач показано на рисунке 3.5. Для задач "выше медианы" доля положительного класса близка к 50% по построению. Задача $SI > 8$ имеет умеренный дисбаланс (35.7% положительных).

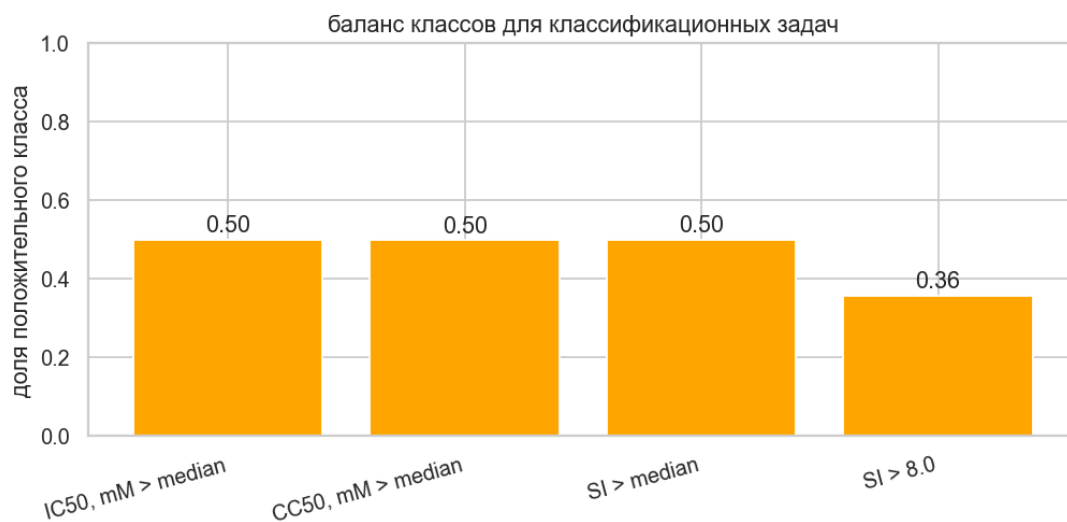


Рис. 3.5: Доля положительного класса для четырёх задач классификации

Вывод. Для $SI > 8$ необходимо учитывать дисбаланс классов: в моделях, поддерживающих параметр `class_weight='balanced'` (лог.регр, случ.лес, лгбм) - он включён, для XGBoost используется параметр `scale_pos_weight`, равный отношению числа отрицательных примеров к положительным. В качестве целевой метрики для подбора гиперпараметров во всех классификационных задачах выбран `roc-auc` - показатель качества бинарного классификатора, который оценивает, как хорошо модель умеет отделять положительный класс от отрицательного при

любом пороге; 0.5 - это типа угадывания, 1.0 - идеальное разделение. Эта метрика слабо чувствительна к дисбалансу классов, поэтому удобна для всех 4х задач.

3.8 Область применимости (Applicability Domain)

Для оценки области применимости моделей рассчитан стандартизованный leverage — ср. кв z-нормированных значений признаков для каждого объекта. Среднее по выборке - 1.00, leverage_p95 — 2.43, 49 соединений за этим пределом (т.е. $49/1001 \approx 4.9\%$ выборки).

Вывод. Предсказания моделей для соединений, попадающих в эту группу — можно считать "ненадёжными".

3.9 Сводка решений по EDA

По результатам EDA приняты следующие проектные решения, применяемые во всех семи моделях:

1. для регрессий целевая переменная преобразуется как $\log(1+y)$;
2. для регрессий применяется фильтрация выбросов по правилу $3 \cdot \text{IQR}$ от таргета;
3. признаковое пространство фильтруется (константы, $|p| > 0.95$, NaN-колонки), оставшиеся пропуски импутируются медианой;
4. для классификационных задач применяется стратифицированный split, а для задачи SI > 8 дополнительно используются веса классов;
5. в качестве линии сравнения включены как линейные, так и древесные модели, чтобы количественно подтвердить нелинейность задачи.

Глава 4

Инструментарий и методология моделирования

4.1 Используемые библиотеки

В работе использовался Python 3.11 и следующие библиотеки: `pandas` и `numpy`, `scikit-learn`; `xgboost` и `lightgbm` — специализированные реализации градиентного бустинга (XGBoost, особенности и параметры в [5]); `optuna` для подбора гиперпараметров через Tree Parzen Estimator [4]; `matplotlib` и `seaborn`, `joblib` для сериализации. Управление окружением осуществляется через менеджер пакетов `uv` с фиксацией версий в `uv.lock`.

4.2 Состав моделей-кандидатов

Выбор моделей обоснован конспектом и расширен градиентным бустингом (3 модели) для сравнения. Для регрессионных задач протестировано семь моделей-кандидатов, для классификационных — шесть. Полный перечень :

Таблица 4.1: Модели

Модель	Регрессия	Классификация
Ridge / Logistic Regression	✓ Ridge	✓ LogReg
ElasticNet	✓	—
K-Nearest Neighbors	✓	✓
Random Forest	✓	✓
Gradient Boosting (sklearn)	✓	✓
XGBoost	✓	✓
LightGBM	✓	✓

Линейные и KNN требуют масштабирования признаков, поэтому собираются через `sklearn.pipeline` с `StandardScaler`.

Все модели с поддержкой балансировки получают параметр `class_weight='balanced'`, для XGBoost дополнительно вычисляется `scale_pos_weight` в задаче с дисбалансом.

4.3 Единый пайплайн обучения

Все семь задач идут по одному алгоритму в методе `run()` абстрактного `BaseTask`.

Шаги пайплайна:

1. загрузка через `data_io.load_raw`;
2. фильтрация признаков (`preprocessing.filter_features`), удаление выбросов по таргету (регрессии), заполнение пропусков медианой, преобразование целевой переменной (`log1p` для регрессий, бинаризация для классификаций);
3. разделение на обучающую и отложенную выборки - 80/20 (для классификаций — стратифицированное по таргету);
4. для каждой модели-кандидата — запуск Optuna: 40, TPE с фиксированным `random=42`; внутри каждого испытания 5-кратная кросс-валидация на обучающей выборке;
5. дообучение лучшей конфигурации каждой модели на полной обучающей выборке и оценка на отложенной;
6. выбор победителя по средней метрике кросс-валидации; сохранение всех артефактов задачи в `artifacts/`.

В качестве метрики для подбора гиперпараметров используется `neg_root_mean_squared_error` для регрессии и `roc_auc` для классификации. Финальная оценка проводится по: MAE, RMSE, R^2 для регрессии; accuracy, precision, recall, F1, ROC-AUC для классификации.

Глава 5

Результаты регрессионных моделей

Сводка по всем регрессионным задачам:

Таблица 5.1: Результаты регрессионных моделей

Задача	Таргет	Лучшая модель	MAE (log)	RMSE (log)	R ²
regression_ic50	IC50	XGBoost	1.016	1.282	0.416
regression_cc50	CC50	Random Forest	0.805	1.087	0.511
regression_si	SI	Random Forest	0.745	0.932	0.171

5.1 Регрессия IC50

Лучшей моделью по итогам кросс-валидации стала **XGBoost**. Гиперпараметры: `n_estimators=256`, `learning_rate=0.023`, `max_depth=4`, `subsample=0.66`, `colsample_bytree=0.82`, `reg_lambda=9.19`. На отложенной выборке достигнуты: MAE=1.016, RMSE=1.28, R²=0.416.

Линейные модели (Ridge, ElasticNet) показали отрицательный R² т.е. фактически оказались хуже чем предсказания средним значением. Ансамбли (Random Forest, GradientBoosting, XGBoost, LightGBM) уверенно решают задачу с R² - 0.34–0.42, но в лидерах XGBoost+RF+GB. LGBM - 0.34

Вывод и рекомендации. Достигнутое качество достаточно для предварительного отбора кандидатов. Возможные улучшения: построение отдельной модели "активен/неактивен" для подвыборки сверхактивных соединений с последующей регрессией внутри подвыборки, ну и конечно стэкинг лучших ансамблевых моделей.

5.2 Регрессия CC50

Лучшая модель – **Random Forest** с конфигом: `n_estimators=660`, `max_depth=23`, `min_samples_split=7`, `min_samples_leaf=2`, `max_features='sqrt'`. Метрики на отложенной выборке: MAE=0.81, RMSE=1.09, R²=0.511.

Регрессия *cc50* оказалась самой предсказуемой из трёх регрессионных задач. Все ансамблевые модели показали близкое качество (R^2 в диапазоне 0.49–0.53), линейные модели конечно хуже - R^2 около 0.28–0.32, но всё равно - положительный R^2 .

Вывод и рекомендации: Модель *CC50* - самая надёжная из трёх и может использоваться как первичный фильтр.

5.3 Регрессия *SI*

Лучшая модель опять **Random Forest** (`n_estimators=528`, `max_depth=16`, `min_samples_split=7`, `min_samples_leaf=4`, `max_features='log2'`). Метрики: MAE=0.74, RMSE=0.93, $R^2=0.171$.

Результат слабее остальных регрессий: *SI* является производной от *IC50* и *CC50* и ошибки исходных таргетов при делении *накапливаются*. Эта гипотеза из EDA, полностью подтверждена количественно: даже лучшая модель объясняет лишь около 17% дисперсии $\log(1+si)$. Все ансамбли показали близкое качество - R^2 около 0.13–0.17, линейные — около R^2 0.07–0.08.

Вывод и рекомендации. Прямую регрессию *si* использовать только как контрольный бейзлайн. В практических задачах *si* получать только как $\text{pred}(\text{CC50}) / \text{pred}(\text{IC50})$ из предсказаний двух более точных.

Глава 6

Результаты классификационных моделей

Сводка по всем классификационным задачам приведена в табл. 6.1. Основная метрика - ROC-AUC. Доп: accuracy, precision, recall, F1.

Таблица 6.1: Результаты классификационных моделей

Задача	Таргет	Лучшая модель	Accuracy	Precision	Recall	F1	ROC-AUC
classification_ic50_median	IC50 > med	Random Forest	0.711	0.675	0.81	0.736	0.779
classification_cc50_median	CC50 > med	GradientBoosting	0.716	0.684	0.8	0.737	0.832
classification_si_median	SI > med	Random Forest	0.657	0.687	0.57	0.623	0.704
classification_si_gt8	SI > 8	XGBoost	0.731	0.667	0.5	0.571	0.744

6.1 Классификация: IC50 выше медианы

Лучшей моделью стала **Random Forest**. На отложенной выборке достигнут ROC-AUC=0.779, F1=0.736. Из шести кандидатов четыре - ансамблевые модели на решающих деревьях (Random Forest, GradientBoosting, XGBoost, LightGBM) показали ROC-AUC - 0.76–0.78, то есть результат устойчив к выбору конкретного ансамбля. Лог. регрессия даёт чуть меньше (около 0.75), но сопоставимый F1 - задача выше/ниже медианы в целом нормально разделяется даже простой линейной моделью.

Вывод. Модель хорошо отделяет два класса: auc - 0.779, f1 - 0.736. Это подтверждает что бинаризация по медиане IC50 - адекватная задача для табличных моделей.

6.2 Классификация: CC50 выше медианы

Лучший результат - **GradientBoosting**: ROC-AUC=0.832, F1=0.737. Это самая удачная из четырёх классификаций: метрики выше остальных, модели ведут себя стабильно.

Вывод. Для бинаризации CC50 по медиане - GradientBoosting - лучший выбор. Имеет смысл смотреть вместе с регрессией CC50: если регрессия и классификация расходятся, объект стоит проверить отдельно.

6.3 Классификация: SI выше медианы

Лучшей моделью стала **Random Forest** с ROC-AUC=0.704 и F1=0.623. Но как и в регрессии - это самая слабая классификация из четырёх. Причина та же: SI считается из IC50 и CC50, поэтому связь с признаками менее прямая, чем у отдельных таргетов.

Вывод. Отдельная классификация SI по медиане даёт скромный результат.

6.4 Классификация: SI > 8

Лучшая модель — **XGBoost** с гиперпараметрами `n_estimators=419`, `learning_rate=0.015`, `max_depth=9`, `subsample=0.71`, `colsample_bytree=0.78`, `reg_lambda=9.21`. Метрики на отложенной выборке: ROC-AUC=0.744, accuracy=0.731, precision=0.667, recall=0.500, F1=0.571.

Здесь умеренный дисбаланс (35.7% с si>8), порог задан в условии.

Ансамбли дают ROC-AUC около 0.73–0.76, но recall ниже (0.47–0.58): признаки ”на границе модель чаще относит классу большинства - типа как ”безопасный” прогноз. Как аналогия: в [7] похожий приём — ML-скрининг на большой выборке с последующей ручной проверкой лучших кандидатов (только там размер выборки - 12 076 365 соединений) .

Вывод и рекомендации. Модель рабочая, но recall слабый. Можно снизить порог вероятности с 0.5 до около 0.3 и попробовать добавить `CalibratedClassifierCV` для калибровки [8], если важнее поймать больше положительных примеров.

Глава 7

Сравнительный анализ и обсуждение

7.1 Регрессия против классификации

Чисто по метрикам, классификации в среднем выглядят сильнее регрессий, что и логично: предсказывать выше/ниже порога – проще, чем предсказывать точное число. Если нужен только отсев или ранжирование по половинам выборки — классификация. Если нужно именно значение таргета – регрессия.

7.2 Линейные модели против ансамблей

Во всех семи задачах древесные ансамбли (Random Forest, XGBoost, GradientBoosting и даже ”слабый”LightGBM) явно превосходят линейные модели по основной метрике (что собственно и говорилось ещё в EDA). Особенно это проявилось в регрессии IC50, где линейные не смогли превзойти даже предсказание средним. Это согласуется с EDA: ни один из 210 признаков по отдельности, таргет почти не объясняет ($|r|_{\max}$ около 0.27), связь скорее нелинейная, соответственно выбор древесных - вполне логичен.

7.3 Согласованность EDA и финальных результатов

Решения по EDA нашли отражение в метриках:

- *Преобразование $\log(1+y)$* – без него линейные модели не работают от слова совсем (отрицательный R^2 даже после логарифмирования, без логарифмирования - ещё хуже из-за выбросов)
- *Удаление коррелированных признаков* - качество не просело после сокращения с 210 до 157-159 признаков, обучение стало быстрее
- *Производный характер SI* – регрессия SI явно слабее CC50 и IC50 что и ожидалось
- *Дисбаланс в задаче SI* > 8 – precision выше recall, типично для дисбаланса даже с весами классов

- *Ансамбли на деревьях* – победили во всех 7 задачах по основной метрике

7.4 Нюансы

- Чуть меньше 5% объектов выбиваются по leverage – для них предсказания менее надёжны
- Регрессия SI по факту дублирует IC50 и CC50: в задании она есть, но лучше считать SI из двух более сильных регрессий
- 1001 строка – не очень много для тяжёлых моделей. На большем датасете, качество скорее всего бы выросло

7.5 Рекомендации по развитию

1. Для $SI > 8$ – CalibratedClassifierCV и подбор порога под нужный баланс precision/recall
2. Стэкинг RF+XGBoost+LightGBM по predict на holdout, где несколько моделей близки по метрике
3. SHAP для лучших моделей RF, XGBoost, LightGBM. В работе не пробовал, но судя обзору [9] для таких моделей есть Tree SHAP, который покажет вклад каждого из 210 дескрипторов в конкретное предсказание и summary plot (что-то типа картинки важности признаков по всей выборке) по отложенной выборке. Dependence plot показал бы взаимодействия между X и Y - типа графика как при значении признака реагирует модель. И что-то типа "правдоподобности правил модели на какие дескрипторы модель опирается и нет ли подозрительных признаков (утечки). Авторы пишут, что при сильной корреляции дескрипторов вклад "размазывается" между признаками. В целом, судя по статье, это именно тот инструментарий, который помог бы разобраться с обученными моделями и понять какие дескрипторы реально "тянут" предсказание

Заключение

В рамках курсовой выполнен полный ML-пайплайн на табличном датасете:

- разобран учебный материал (конспект студентов, статьи на Хабре) по классическому МО и подбору гиперпараметров
- сделаны EDA и предобработка: $\log 1p$ для регрессий, фильтрация признаков, $3 \cdot IQR$, стратификация, веса классов для $SI > 8$
- сравнено 6-7 моделей на задачу, Optuna(TPE, 5fold, 40trials)
- итоговые метрики по тестовой части: лучшие R^2 — CC50 0.51 (RF), IC50 0.42 (XGBoost), SI 0.17 (RF), классификации ROC-AUC 0.70–0.83
- даны рекомендации как можно улучшить, если взяться за задачу всерьёз (калибровка, стэкинг, SHAP)
- собрано Python-приложение: src-layout, pyproject.toml, "растовский"uv, единый пайплайн на семь задач, логирование. Хотя где-то можно пилить, как хочешь и реализовывать "красивую" архитектуру, а не "абы-как, лишь бы было сделано вчера, а заказчику - прокатит, что-нибудь придумаем".
- написан отчёт в TexShop на движке XeLaTeX:) и вряд ли кто-то до сюда дочитает, поэтому можно откровенно признаться: я "заманался" с этим заданием.

Полный исходный код, данные, графики EDA, модели и метрики - в репозитории:

<https://github.com/stepkar/chemical-coursework>.

Литература

- [1] **Creative Diagnostics**. CC50/IC50 Assay for Antiviral Research. — Электронный ресурс: <https://antiviral.creative-diagnostics.com/cc50-ic50-assay.html>.
- [2] **TeachInMSU**. ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ В ХИМИИ И МАТЕРИАЛОВЕДЕНИИ (МИТРОФАНОВ АРТЕМ АЛЕКСАНДРОВИЧ и др.: конспект курса лекций.) Химфак МГУ
- [3] **Luciano Ramalho** Fluent Python: Clear, Concise, and Effective Programming. — 2nd edition — O'Reilly, 2022.
- [4] **badcasedaily1**. Библиотека Optuna в Python для оптимизации гиперпараметров. — Хабр, блог компании OTUS, 2024. — <https://habr.com/ru/companies/otus/articles/801463/>.
- [5] **XGBoost: особенности работы**. — Хабр, 2024. — <https://habr.com/ru/articles/799725/>.
- [6] **Гайд по Scikit-learn в 2025: собираем пайплайн, который не сломается**. — Хабр, блог компании Нетология, 2025. — <https://habr.com/ru/companies/netologyru/articles/911216/>.
- [7] **ИИ помог обойти защиту резистентных бактерий: открыт новый класс перспективных антибиотиков**. — Хабр, блог компании BotHub, 2024. — <https://habr.com/ru/companies/bothub/articles/788170/>.
- [8] **m0rtido**. Варим ML Boot Camp III: Starter Kit. — Хабр, 2017. — <https://habr.com/ru/articles/324924/>.
- [9] **boygenius13**. Интерпретация моделей и диагностика сдвига данных: LIME, SHAP и Shapley Flow. — Хабр, блог компании Open Data Science, 2022. — <https://habr.com/ru/companies/ods/articles/599573/>.